EMBEDDED LINUX LAB 02

Character Device Driver, procfs and sysfs
Linux 5.15 | ARMv7-A | QEMU Versatile Express

Course	LOS201 - Embedded Linux Systems
Target	ARM Cortex-A9 / vexpress-a9 / 512 MiB / 2 vCPUs
Kernel module	lab2_driver.ko | Major 240, Minor 0
Primary evidence	logs/lab2_full_test.log (uploaded student run)
Report date	20 October 2026

Verification scope
The uploaded log directly verifies QEMU boot, automatic module registration, device-node creation and read/write tests. procfs/sysfs were manually demonstrated in the conversation, and the user reported the updated procfs/sysfs script passed; that automated run is not contained in the uploaded log.

Prepared from the supplied Lab 02 source code, boot artifacts and terminal evidence. No independent fresh build or additional hardware experiments are claimed.

1. Objective and experiment scope

Develop and verify a loadable Linux Character Device Driver on an emulated ARM Cortex-A9 board. The exercise demonstrates the complete connection from shell commands to kernel file operations, kernel-maintained state, procfs statistics, and read-only sysfs attributes.

1.1 Deliverables and interfaces

Component	Function	Location
Character device	Open, read, write, close	/dev/lab2
procfs	Human-readable driver state	/proc/lab2_info
sysfs	buffer_len, open_count, last_data	/sys/class/lab2_class/lab2/
Test scripts	Automated checks with comparisons	/usr/bin/test_driver.sh; /usr/bin/test_procfs.sh
Loadable module	ARMv7 ELF module	/lib/modules/5.15.0/lab2_driver.ko

1.2 Development environment

Setting	Value confirmed by submitted materials
Emulated platform	QEMU vexpress-a9; Cortex-A9; 512 MiB RAM; 2 requested CPUs
Kernel	Linux 5.15.0; ARMv7l; PL011 console ttyAMA0
Root filesystem	BusyBox for ARM 32-bit, statically linked; gzip-compressed cpio initramfs
Module	lab2_driver.ko: ELF 32-bit ARM EABI5, kernel module
Toolchain prefix	arm-linux-gnueabihf- (defined in Makefile)
Build tree	~/embedded_lab1/kernel/linux-5.15 (defined in Makefile)

Scope boundary
This Lab 02 run uses QEMU direct kernel boot (-kernel, -dtb and -initrd). U-Boot is not part of the demonstrated execution path. MTD/JFFS2 is outside the evidence available in this submitted archive.

2. System architecture and startup

The architecture below is based on the submitted driver and initramfs startup files. The three user-facing interfaces share one kernel buffer and counters.

2. System architecture (continued)

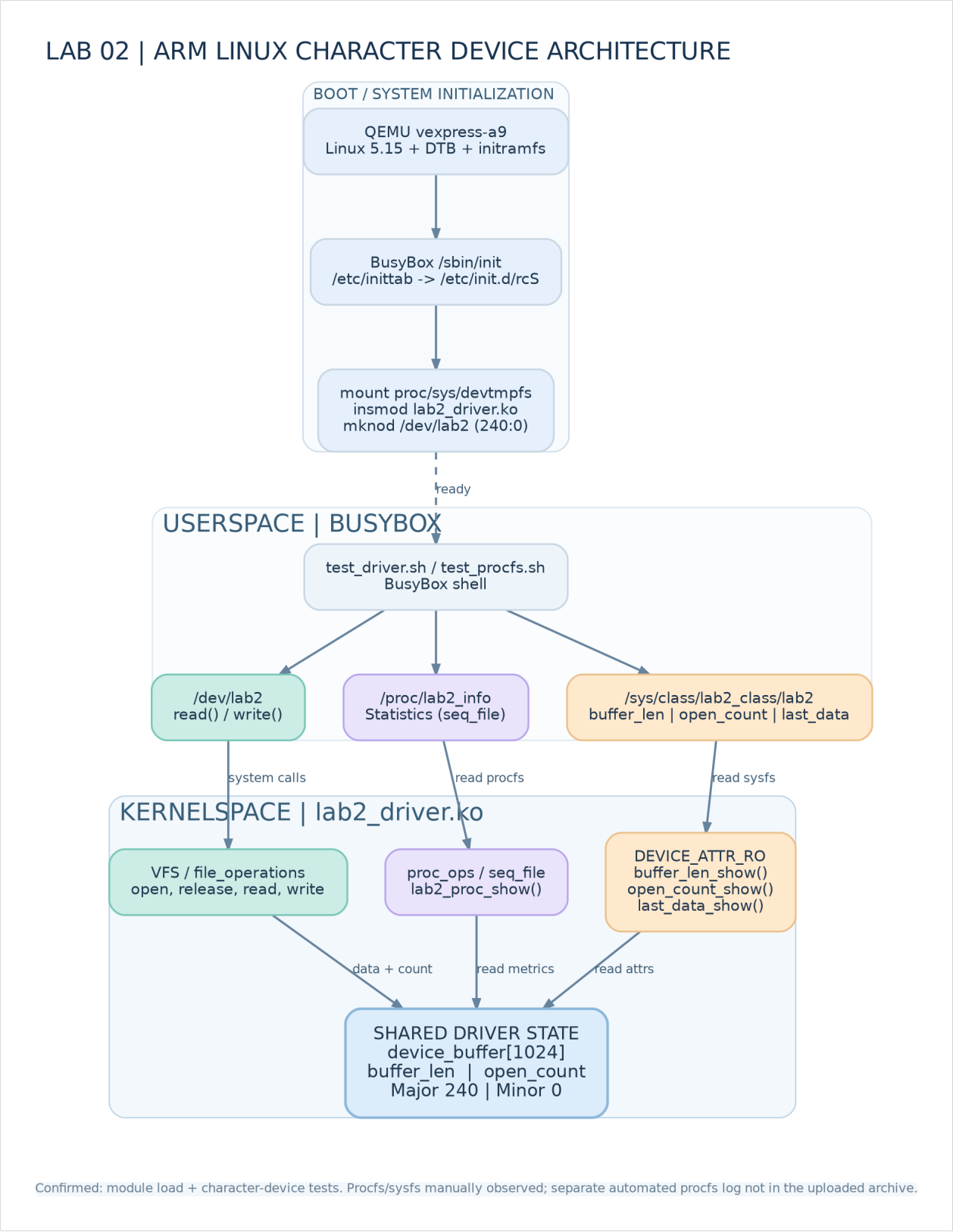


Figure 1. QEMU boot path, device/procfs/sysfs user interfaces and shared state implemented by lab2_driver.ko.

3. Implementation details

3.1 Kernel module registration

The module reserves the fixed major number 240 using register_chrdev(), creates class lab2_class and device lab2, exports three sysfs attributes through sysfs_create_group(), and publishes /proc/lab2_info using proc_create(). It handles open/read/write/release through struct file_operations.

```
#define DEVICE_NAME  &amp;quot;lab2&amp;quot;
#define CLASS_NAME   &amp;quot;lab2_class&amp;quot;
#define BUFFER_SIZE  1024
#define DRIVER_MAJOR 240

static char device_buffer[BUFFER_SIZE];
static int buffer_len = 0;
static int open_count = 0;
```

3.2 Read/write semantics

Writes use copy_from_user() and cap the transferred length at 1,023 bytes, reserving space for a NUL terminator. The shared buffer is overwritten by each write. Reads use copy_to_user() and advance the file offset. After all currently valid bytes have been read, the driver returns 0 (EOF), allowing cat to terminate.

```
# Read/write from the guest shell
echo &amp;quot;Hello from userspace, LAB-02!&amp;quot; &amp;gt; /dev/lab2
cat /dev/lab2
```

Because echo adds a newline, this example yields 30 bytes in the submitted run. The driver is a last-message buffer rather than a persistent stream or a regular seekable file.

3.3 Device initialization

BusyBox /sbin/init reads /etc/inittab, which launches /etc/init.d/rcS. rcS mounts virtual filesystems, sets hostname embedded-lab2, inserts lab2_driver.ko, then creates /dev/lab2 as a character node (240:0). The procfs and sysfs endpoints are registered by the module itself.

```
::sysinit:/etc/init.d/rcS
::askfirst:-/bin/sh

# Core actions from rcS:
insmod /lib/modules/5.15.0/lab2_driver.ko
mknod /dev/lab2 c 240 0
chmod 666 /dev/lab2
```

> **Practical warning**
> rcS automatically inserts lab2_driver.ko. Running insmod again produces File exists, which is a duplicate-load condition, not a failure of the already loaded driver. The revised test scripts check /proc/modules first.

4. Reproduce the tested boot

From Ubuntu, use the known-good Linux kernel and DTB from Lab 01 together with the submitted Lab 02 initramfs. Run the following command in ~/LOS201/embedded_lab2:

```
qemu-system-arm -M vexpress-a9 -cpu cortex-a9 \
  -m 512M -smp 2 -nographic \
  -kernel ../embedded_lab1/output/zImage \
  -dtb ../embedded_lab1/output/vexpress-v2p-ca9.dtb \
  -initrd output/initramfs_lab2.cpio.gz \
  -append &#x27;console=ttyAMA0,115200 rdinit=/sbin/init mem=512M&#x27;
```

When the guest presents / #, run lsmod, ls -l /dev/lab2 and cat /proc/lab2_info. Press Ctrl+A then X to exit QEMU serial mode.

5. Verification procedure and actual evidence

5.1 Recorded QEMU/driver state

Checkpoint	Recorded observation	Status
Kernel/target	Linux 5.15.0 on armv7l; hostname embedded-lab2	PASS
Module insertion	lab2_driver: loaded, major=240	PASS
Device node	/dev/lab2 is character node 240,0	PASS
Character write/read	Hello from userspace, LAB-02! read back identically	PASS
Repeated read/write	Message_1, Message_2, Message_3 comparisons pass	PASS
Script result	DRIVER_TEST_EXIT_CODE=0	PASS
procfs and sysfs	Manual output observed during conversation	Observed separately
Automated procfs test	User reported PASS; no separate output in supplied log	Not logged

5.2 Evidence excerpts from the uploaded log

```
lab2_driver: /proc/lab2_info created
lab2_driver: loaded, major=240
[ LAB-02 ] lab2_driver loaded OK
[ LAB-02 ] /dev/lab2 created (major=240, minor=0)

PASS: WRITE completed
PASS: READ data matches WRITE data
PASS: Message_1
PASS: Message_2
PASS: Message_3
=== CHARACTER DRIVER TEST PASSED ===
DRIVER_TEST_EXIT_CODE=0
```

The log also records /proc/lab2_info after the tests: Buffer size 1024 bytes, Data length 10 bytes, Open count 8, and Last data Message_3. The resulting length of 10 includes the newline added by echo.

5.3 Separate procfs/sysfs observation

In the conversation, the user displayed /proc/lab2_info together with sysfs values buffer_len=30, open_count=8, and last_data=Hello from userspace, LAB-02!. These values are consistent with the demonstrated device writes and reads. The user later confirmed that the updated procfs/sysfs test script passed, but the uploaded log contains no PROCFS_TEST_EXIT_CODE marker.

Evidence follow-up
For a single auditable submission log, run both scripts in one fresh QEMU session and record DRIVER_TEST_EXIT_CODE=0 and PROCFS_TEST_EXIT_CODE=0. Do not add a PASS line manually to existing evidence.

6. Troubleshooting and checkpoints

Symptom	First check	Recommended action
No QEMU command	command -v qemu-system-arm	Install QEMU on Ubuntu host.
Missing zImage / DTB	ls ../embedded_lab1/output/	Run from the correct folder; inspect paths.
No / # prompt	Console, initramfs, rdinit	Check ttyAMA0 and /sbin/init inside cpio.
insmod File exists	lsmod | grep lab2_driver	Module is already loaded; skip reinsertion.
insmod No such file on host	Host versus guest prompt	Run insmod inside QEMU (/ #).
Invalid module format	uname -r; module vermagic	Rebuild against matching kernel/config.
Device node missing	ls -l /dev/lab2; /proc/devices	Check module registration and 240:0.
procfs/sysfs missing	lsmod; mount; dmesg	Check proc/sys mounts and registration.
Edited script unchanged	cpio -t; chosen -initrd image	Repackage and boot new archive.
Wrong byte count	echo versus printf	Remember echo adds newline.

6.1 Minimal QEMU guest validation commands

```
uname -a
lsmod
ls -l /dev/lab2
cat /proc/lab2_info
cat /sys/class/lab2_class/lab2/buffer_len
cat /sys/class/lab2_class/lab2/open_count
cat /sys/class/lab2_class/lab2/last_data
sh /usr/bin/test_driver.sh; echo &amp;quot;DRIVER_TEST_EXIT_CODE=$?&amp;quot;
sh /usr/bin/test_procfs.sh; echo &amp;quot;PROCFS_TEST_EXIT_CODE=$?&amp;quot;
```

7. Limitations and recommendations

Synchronization: lab2_write() and lab2_open() use lab2_mutex, but lab2_read() and procfs/sysfs show routines inspect shared state without equivalent locking. Concurrent reads/writes may observe inconsistent data. This is a code-review observation; no concurrent stress test is recorded.

Error paths: sysfs_create_group() and proc_create() may fail without causing lab2_init() to fail. A more robust driver would propagate required interface failures and unwind registered resources consistently.

Hard-coded resources: major 240 and the global 1024-byte buffer suit the exercise but have limitations. Module and test scripts depend on the same major, kernel release and paths. These choices would need refinement for a reusable production driver.

Untested scope: the supplied evidence does not establish MTD/JFFS2 functionality, stress behavior, alternative kernels, or deployment on physical hardware.

8. Conclusion

The supplied Lab 02 system booted successfully in QEMU and loaded the ARM kernel module at major 240. The recorded tests demonstrate correct character-device write/read behavior, including three repeated payload comparisons and a zero script exit status. The procfs and sysfs interfaces were separately observed working, while the combined automated procfs/sysfs result should be recorded into the repository log to strengthen reproducibility.

Reference files in repository

Source: driver/lab2_driver.c; driver/Makefile. **Runtime:** rootfs/initramfs/etc/init.d/rcS and /etc/inittab. **Tests:** rootfs/initramfs/usr/bin/test_driver.sh and test_procfs.sh. **Evidence:** logs/lab2_full_test.log. **Tutorial:** README.md. **Architecture:** diagrams/Lab02_Character_Driver_Architecture.pdf.